



**WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
MASTER STUDY PROGRAM: CYBERSECURITY**

MASTER THESIS

SUPERVISOR:
Dr. Darius Galis

GRADUATE:
Paul - Andrei Abrudan

**TIMIȘOARA
2026**

WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
MASTER STUDY PROGRAM: CYBERSECURITY

Automated Security Analysis of the Rust Crate Ecosystem: A Distributed Framework for Vulnerability and Malicious Code Detection

SUPERVISOR:
Dr. Darius Galis

GRADUATE:
Paul - Andrei Abrudan

TIMIȘOARA
2026

Abstract

The Rust programming language has gained popularity in multiple programming fields due to its memory safety features and performance. Rust's package ecosystem, crates.io, delivers over 150,000 crates to the language's users. Due to the rapid growth of the language and ecosystem, potential security risks are present, including known vulnerabilities in dependencies, a history of supply chain attacks through code injection and typosquatting, secrets being leaked in source code and cases of malicious build scripts present in the optional build.rs file at the root of the packages. This thesis presents a distributed framework for automating security analysis on the Rust package library ecosystem. The system parses metadata from the crates.io database dump, constructing a dependency graph and applying multiple analysis modules: Vulnerability scanning using 'cargo audit', which uses the RustSec Advisory Database, Secret leakage detection using the FOSS tool GitLeaks, heuristic analysis for the 'build.rs' script, detecting network calls, process creation, obfuscation and suspicious TLDs used, the possibility of LLM code scoring using LM Studio's REST API, typosquatting detection using multiple string similarity algorithms and transitive dependency chain risk propagation. Horizontal scaling is possible across multiple nodes, achieved with Redis used as a work queue, with the possibility of upgrading to a faster in-memory database. The results are persisted in a PostgreSQL database and can be visualized through the web dashboard. This thesis aims to make it easier to search for cases of malicious dependencies in the Rust ecosystem, providing tools for security auditing.

Keywords: crates.io, Rust, supply chain security, typosquatting, distributed systems, software security

Statement of Scientific integrity

I declare that this BSc / MSc thesis is an original report and represents my own work, conducted under supervision of [name(s) supervisor(s)]. All research methods, data collection, analysis and interpretation presented in this thesis are accurately reported and all sources have been cited appropriately where I have made use of ideas and/or visuals of others.

If you did not use generative AI tools: I did not use generative AI during the writing and research process.

If you used generative AI tools: I used generative AI during the writing and research process. The details are in the Appendix A.

Contents

1	Introduction	7
1.1	Context and Motivation	7
1.2	Problem Statement	8
1.3	Objectives	8
1.4	Scope and Delimitations	8
1.5	Thesis Structure	9
2	State of the art	11
2.1	Resources	11
2.2	Software Supply Chain Security	12
2.3	Package Ecosystem Vulnerabilities	12
2.4	Typosquatting in Package Registries	12
2.5	Malicious Code Detection Techniques	12
2.6	Build Script Security Risks	12
2.7	Large Language Models for Code Analysis	12
2.8	Existing Tools and Related Work	12
2.9	Summary and Research Gap	12
3	Methodology	13
3.1	Research Methodology	13
3.2	Data Collection	13
3.3	Analysis Framework Design	13
3.4	Evaluation	13
4	System Architecture	15
4.1	High-Level Architecture Overview	15
4.2	Data Pipeline	15
4.3	Dependency Graph Creation	15
4.4	Distributed Work Queue	15
4.5	Database Schema Design	15
4.6	Web Dashboard	15
4.7	Deployment and Infrastructure	15
5	Analysis Modules	17
5.1	Cargo Audit - Vulnerability Scanning	17
5.2	Gitleaks - Secret Detection	17
5.3	Build Script Analysis	17
5.4	LLM-Assisted Malicious Code Detection	17
5.5	Typosquatting Detection	17

5.6	Transitive Dependency Risk Propagation	17
6	Implementation Details	19
6.1	Technology Stack	19
6.2	CLI Application Structure	19
6.3	Redis Queue Implementation	19
6.4	Cycle Detection and Resolution	19
6.5	Worker Lifecycle and Fault Tolerance	19
6.6	Batch Processing and Database Optimization	19
7	Results and Evaluation	21
7.1	Dataset Overview	21
7.2	Vulnerability Landscape	21
7.3	Secret Leakage Findings	21
7.4	Build Script Risk Assessment	21
7.5	LLM Malicious Code Analysis	21
7.6	Typosquatting Detection Results	21
7.7	Dependency Chain Risk Propagation	21
7.8	Performance and Scalability	21
8	Discussion	23
8.1	Interpretation of Results	23
8.2	Comparison with Existing Approaches	23
8.3	Limitations	23
8.4	Threats to Validity	23
9	Conclusions and Future Work	25
9.1	Summary of Contributions	25
9.2	Future Directions	25
10	Some Latex Commands	27
	Bibliography	27
A	Appendix - Generative AI Tools Usage	29
A.1	ChatGPT Details	29
A.2	Deep L Details	30
A.3	Microsoft Copilot	30

Chapter 1

Introduction

1.1 Context and Motivation

All relatively modern software relies on thirdparty packages, distributed through different centralized registries. The Rust programming language has grown from it's initial release by Mozilla back in 2015, to today where crates.io hosts over 150.000 crates with multiple billions of downloads, being the pillar of the Rust library supply chain.

This growth can be a double-edged sword, while being beneficial for productivity, it introduces an immense attack surface, which remains underexplored as of now. In most languages, including Rust, when a developer adds a dependency to their project, they not only trust the direct dependency but also all its entire dependency tree, which can contain hundreds if not thousands of packages. A single compromised dependency, wether due to an unpatched vulnerability, a typosquat or due to an adversary takeover, can propagate malicious code across a wide part of the ecosystem.

Relatively recent incidents in other package ecosystems, such as npm's attack in 2018 known as 'event-stream', 'ua-parser-js' from 2021 and the 'colors.js' and 'faker.js' sabotages in 2022, demonstrated that supply chain attacks are practical attacks, very effective and can become extremely complex. Rust's memory safety gurantees mitigate some classes of vulnerabilities (such as buffer overflows), they provide no help against attacks such as memory-safe malicious code and secret exfiltration from the environment.

Another attack vector specific to some languages, including Rust, is that the build system allows arbitrary code execution by crates at compile time, using 'build.rs' scripts. The scripts can perform operations such as network communication, process execution and have file system access. This mechanism is essential for some usecases such as native library linking and code generation, however it also is a dangerous attack vector that can infect a machine just compiling a malicious crate, without ever executing the resulted binary.

The motivation comes from the observation that some security tools already exist for specific Rust crate analysis, however no ecosystem-wide comprehensive framework exists, that integrates multiple modules and can apply them at scale.

1.2 Problem Statement

Rust’s crate ecosystem, lacks a scalable analysis framework that is capable of performing module-based scanning, first, the scale of crates.io is massive, with over 150.000+ crates, any ecosystem-wide analysis requires a scalable and distributed architecture, capable of processing crates efficiently. Individual tools address specific parts of security analysis, such as secrets scanning and existing known vulnerabilities. There is no existing solution that combines build script analysis, malicious code detection, typosquatting crate name analysis, vulnerability scanning and secret detection into a complete comprehensive solution. It is also the case that a crate with no direct vulnerabilities can still pose a risk if it exports an API dependent on a vulnerable package, tracking these risks is still an existing challenge. The ‘build.rs’ script allows arbitrary code execution at compile time and it is the case that no study has tracked possibly suspicious build script behaviours across the entire crates.io database. While typosquatting has been studied in other package libraries such as npm and PyPI, the Rust’s ecosystem has seen no studies regarding this. Large Language Models have demonstrated capabilities in understanding code, however the deployment of LLM analysis at scale has not been explored over package libraries.

1.3 Objectives

The objective of this thesis is to design, implement and evaluate a toolkit facilitating comprehensive distributed security analysis over the Rust’s crate ecosystem, the specific objectives are:

Designing a modular toolkit, which unifies multiple modules and analysis techniques into a single pipeline, enabling user friendly security assessment of crates published on crates.io Implementing distributed processing over the entire toolkit, using a ”in-memory database”-based work queue, together with topological ordering of the crates dependencies, we enable horizontal scaling across workers and achieving acceptable throughput on consumer hardware. Develop and integrate multiple analysis modules such as: vulnerability scanning using RustSec’s Database, secret detection in available source code, heuristic ‘build.rs’ analysis for detecting unusual behaviours such as network communication, process execution, obfuscated/encrypted code and suspicious TLD usage, LLM-based malicious code scoring, typosquatting detection using multiple similarity algorithms (evenshtein, Damerau-Levenshtein, Jaro-Winkler, keyboard proximity distance, and prefix), and dependency chain propagation analysis. Conduct an entire ecosystem-wide analysis, either by running the framework on a subset of the crates.io registry, or the entire database, characterizing and doing manual analysis on provided warnings. Develop a web-based dashboard for user friendly exploration and visualization of analysis results. Evaluate the framework’s performance and scalability, measuring throughput, bottlenecks and possible improvements.

1.4 Scope and Delimitations

TODO: Formatting, cleaning, typos, etc

In scope:

- Analysis of multiple crates published on crates.io, imported from the official database dump
- Multiple analysis modules as described in the objective list
- Analysis of crate source code from the git repository
- Having distributed processing capabilities using an in-memory database for work distribution and PostgreSQL for resulting data
- A web dashboard for visualization
- Performance evaluation of analysis and of found problems and exploits

Out of scope:

- Dynamic analysis or runtime behavior
- Analysis of compiled binaries
- Manual verification of all results, due to the framework being designed as a screening tool, requiring human validation
- Real-time monitoring
- Analysis of other package ecosystems
- Formal proofs for detection correctness

1.5 Thesis Structure

This thesis is organized in this way:

Chapter 2 - State of the art: Is a review about existing research and literature on software supply chain security, malicious code detection techniques, package vulnerabilities, typosquatting and build scripts used maliciously. It identifies a research gap related to the previously mentioned objectives.

Chapter 3 - Methodology: Describes the research, data collection and importing, framework design, analysis modules and the evaluation metrics used.

Chapter 4 - System architecture: We detail the high level architecture, covering the data importing, dependency graph creation, the design of the work queue, database schema, the dashboard and deployment infrastructure and optimization scripts.

Chapter 5 - Analysis Modules: We detail the analysis modules used, algorithms, heuristics, fault tolerance and optimization.

Chapter 6 - Implementation Details - We present the technology stack, the CLI app usage, the queue implementation and database implementation

Chapter 7 - Results & Evaluation - This chapter is about the findings resulted by running the framework, vulnerability, leaked secrets, build system statistics, typosquatting possibilities, dependency chain risks and performance benchmarks.

Chapter 8 - Discussion: We interpret the results, compare them to existing literature, discuss limitations and address threats to validity.

Chapter 9 - Conclusion & Future Work: We present a summary of the contribution of this thesis and state possible directions for future research or development

Chapter 2

State of the art

The initial stage involves investigating existing work in the area of research to establish the foundation for the thesis.

2.1 Resources

Scientific articles can be found in different online databases:

- ACM Digital Library
- Science Direct
- IEEE Explore
- Annual Reviews
- Springer Link
- Wiley InterScience
- SCOPUS
- Google Scholar
- JSTOR (social sciences, arts & humanities)

Identify communities on the topic (specific workshops on national and international conferences).

Some of the articles from the databases are accessible through university IP.

- 2.2 Software Supply Chain Security**
- 2.3 Package Ecosystem Vulnerabilities**
- 2.4 Typosquatting in Package Registries**
- 2.5 Malicious Code Detection Techniques**
- 2.6 Build Script Security Risks**
- 2.7 Large Language Models for Code Analysis**
- 2.8 Existing Tools and Related Work**
- 2.9 Summary and Research Gap**

Chapter 3

Methodology

3.1 Research Methodology

3.2 Data Collection

3.3 Analysis Framework Design

3.4 Evaluation

Chapter 4

System Architecture

4.1 High-Level Architecture Overview

4.2 Data Pipeline

4.3 Dependency Graph Creation

4.4 Distributed Work Queue

4.5 Database Schema Design

4.6 Web Dashboard

4.7 Deployment and Infrastructure

Chapter 5

Analysis Modules

5.1 Cargo Audit - Vulnerability Scanning

5.2 Gitleaks - Secret Detection

5.3 Build Script Analysis

5.4 LLM-Assisted Malicious Code Detection

5.5 Typosquatting Detection

5.6 Transitive Dependency Risk Propagation

Chapter 6

Implementation Details

6.1 Technology Stack

6.2 CLI Application Structure

6.3 Redis Queue Implementation

6.4 Cycle Detection and Resolution

6.5 Worker Lifecycle and Fault Tolerance

6.6 Batch Processing and Database Optimization

Chapter 7

Results and Evaluation

7.1 Dataset Overview

7.2 Vulnerability Landscape

7.3 Secret Leakage Findings

7.4 Build Script Risk Assessment

7.5 LLM Malicious Code Analysis

7.6 Typosquatting Detection Results

7.7 Dependency Chain Risk Propagation

7.8 Performance and Scalability

Chapter 8

Discussion

8.1 Interpretation of Results

8.2 Comparison with Existing Approaches

8.3 Limitations

8.4 Threats to Validity

Chapter 9

Conclusions and Future Work

9.1 Summary of Contributions

9.2 Future Directions

Chapter 10

Some Latex Commands

In this chapter are presented some useful Latex commands that could be used in thesis written part.

Appendix A

Appendix - Generative AI Tools Usage

An overview how generative AI Tools are used in this thesis is presented in Table A.1.

AI Tool	Use case	Used in	Remarks
ChatGPT 5.0	Generate functionality list	Chapter 1, section 1.3	Prompts in section A.1
	Generate initial state of the art bibliography list	Chapter 2	Prompts in section A.1
Microsoft Copilot	Code generation	Chapter 4 pp.2-21	Details in section A.3
Deep L	Translation of text pages	Chapter 2	Prompts in section A.2
Grammarly	Correct grammar and style mistakes	Entire work	Free version
...			

Table A.1: Overview of generative AI Usage

A.1 ChatGPT Details

Specify for each usage the following:

1. Purpose of use
2. Method (which prompts and for which parts of the thesis)
3. How the output has been examined/verified (has the generated output been examined for accuracy and reliability?)
4. How the output is implemented/used

A.2 Deep L Details

A.3 Microsoft Copilot